

Reproducing and Extending PTXPhish:

A Lightweight Calldata-Based Detector for Payload-Based Transaction Phishing on Ethereum

Course Project - Research Paper Reproduction Report

Adityaraj Shyamsundar Bhandari

Student ID: 2024UCP1639

Branch: CSE

GitHub: github.com/Adityaraj1005/PTXPhish-Reproduction-1639

Reproduction target: Z. Chen, Y. Hu, B. He, D. Luo, L. Wu, Y. Zhou, *Dissecting Payload-based Transaction Phishing on Ethereum*, NDSS Symposium 2025.

October 2026

Abstract

Phishing on Ethereum has moved beyond fake websites and simple fund transfers. In *payload-based transaction phishing* (PTXPhish), the scammer gets the victim to sign a real transaction whose input data (calldata) quietly hands assets or permissions to the attacker. The NDSS 2025 paper by Chen et al. studied this threat, grouped it into four categories (ice phishing, NFT order scams, address poisoning and payable function scams), and built a rule-based detector that reports an F1-score above 99% using a local 2 TB archive node and full transaction-history replay. In this project we reproduce the spirit of that study on a student budget. We build a deterministic, single-pass classifier that reads only the 4-byte function selector, the native ETH value and a few argument words of the calldata, fetched through a pool of free public RPC endpoints with automatic failover. On a balanced benchmark of 500 transactions (125 per class) our first baseline reached 88.60% accuracy (macro F1 0.887). By studying the confusion matrix and the selectors behind every error, we extended the selector registry and added a small number of carefully ordered rules, reaching 98.00% accuracy (macro F1 0.980) on the same sample and 97.60% (macro F1 0.976) on a second random sample. We also built a Streamlit dashboard, an on-chain alert engine that follows Section V-D of the paper, and a pre-signing checker that explains a transaction in plain English before the user signs it and warns about lookalike addresses. We discuss honestly why a calldata-only method cannot reach 100%, and we describe a dataset-labelling concern we found while analysing the remaining errors. All code, data-processing scripts and result files are in our public repository.

Keywords: Ethereum, phishing detection, calldata analysis, function selectors, address poisoning, ice phishing, reproducibility.

Contents

1. Introduction
 2. Background
 3. Summary of the Original Paper
 4. Dataset and Data Engineering
 5. System Design
 6. Implementation Details
 7. Experimental Setup
 8. Results
 9. Comparison with the Original Study
 10. Dashboard, Alert Engine and Pre-Sign Checker
 11. Limitations and Threats to Validity
 12. Future Work
 13. Conclusion
- References
- Appendix A: Selector Registry
- Appendix B: Reproduction Guide
- Appendix C: Glossary

1. Introduction

Ethereum is more than a currency. It is a computer that anybody can program, and most of the value moved on it today flows through smart contracts such as token contracts, decentralized exchanges and NFT marketplaces. Because these contracts are complicated, a normal user cannot read what a wallet pop-up is really asking them to sign. Attackers have learned to exploit exactly this gap. Instead of stealing a password, a modern scammer builds a transaction whose hidden parameters give the attacker permission to move the victim's tokens, or that sends the victim's ETH straight to a contract the attacker controls. The victim clicks "Confirm" and the blockchain faithfully executes what was signed.

Chen et al. called this family of scams **payload-based transaction phishing**, or PTXPhish, and presented the first systematic study of it at NDSS 2025 [1]. They collected a ground-truth dataset, defined four main categories of attack, designed rules that look at contract code, input data, addresses and transaction history, and then ran the detector over 300 days of live Ethereum data. The results are impressive, but the method depends on infrastructure that a student cannot easily obtain: a local archive node of roughly 2 TB and a transaction-replay engine.

1.1 Motivation and Problem Statement

Our course assignment was to reproduce the paper. While reading it, we noticed that most of the four attack categories leave a very visible footprint in the *first four bytes* of the transaction input, the function selector. An `approve` call always starts with `0x095ea7b3`, a token `transfer` with `0xa9059cbb`, and a Seaport order fulfilment with its own fixed selector. This led to a simple question: **how far can a purely static, calldata-based classifier go, compared with the heavy rule engine of the original paper?** If the answer is "far enough", then wallets, browser extensions and student projects could run such a check in milliseconds with nothing more than a public RPC endpoint.

1.2 Objectives

- Understand the PTXPhish taxonomy and detection rules of the original paper and re-create its data pipeline.
- Build a unified detector that classifies a transaction into one of four classes using only information available from a normal (non-archive) RPC call.
- Evaluate it on a balanced, stratified sample and improve it using error analysis rather than guesswork, with a target of at least 95% accuracy.
- Turn the research prototype into something usable: a dashboard, a risk score, a revoke link and an on-chain alert generator.
- Document honestly what the method can and cannot do.

1.3 Contributions

- A reproducible pipeline (cleaning, RPC access with failover, four detectors, one unified evaluator) published at the repository listed on the title page.
- A decision cascade whose accuracy rose from 88.60% to 98.00% on a 500-transaction balanced benchmark, with a full audit trail of which change fixed which errors.
- An error analysis that identifies the structural limits of calldata-only classification (identical calldata with different labels, dust-versus-real amount overlap).
- A dashboard (`app.py`) and an alert engine (`alert_engine.py`) that implement the paper's real-time mitigation idea.
- A pre-signing checker (`presign.py`) that reads calldata pasted from a wallet pop-up, explains it in plain English, gives a risk level, and compares two addresses for the address-poisoning trick. It runs offline and has 14 unit tests.

- A discussion of a labelling issue in our cleaned dataset that we noticed late in the project, together with a plan to fix it.

1.4 Report Organisation

Section 2 gives the blockchain background needed to follow the rest. Section 3 summarises the original paper. Sections 4 to 7 describe our data, design, implementation and experimental setup. Section 8 presents results and error analysis, Section 9 compares with the paper, Section 10 describes the dashboard, and Sections 11 to 13 cover limitations, future work and conclusions. The appendices contain the selector registry, a reproduction guide and a glossary.

2. Background

2.1 Accounts, Transactions and Calldata

Ethereum has two kinds of accounts: externally owned accounts (EOAs), controlled by a private key, and contract accounts, controlled by code. A transaction is a signed message from an EOA. Its most important fields for this project are `from`, `to`, `value` (the amount of native ETH, in wei) and `input` (also called calldata or payload). When `to` is a contract, the input tells the contract which function to run and with which arguments.

2.2 Function Selectors and ABI Encoding

The Solidity ABI defines the first four bytes of calldata as the **function selector**: the first 4 bytes of the Keccak-256 hash of the function signature, for example `transfer(address,uint256)`. Each argument that follows is padded to a 32-byte word. In hexadecimal text, the input of a standard ERC-20 `transfer` is therefore 2 characters for the 0x prefix, 8 characters for the selector and two words of 64 characters, which is 138 characters in total. A `transferFrom` carries three words and has 202 characters. These lengths showed up repeatedly in our diagnostics and helped us sanity-check the parser.

Table 1. Key selectors and their meaning (ABI-encoded calldata layout)

Selector	Function	Words	Role in PTXPhish
0x095ea7b3	<code>approve(address,uint256)</code>	2	Ice phishing: grants a spending allowance
0xa22cb465	<code>setApprovalForAll(address,bool)</code>	2	Ice phishing: approves a whole NFT collection
0xd505accf	<code>permit(...)</code>	7	Ice phishing: off-chain signed allowance
0x23b872dd	<code>transferFrom(address,address,uint256)</code>	3	Ice phishing drain or zero-value poisoning
0xa9059cbb	<code>transfer(address,uint256)</code>	2	Address poisoning (dust or zero transfers)
0x42842e0e	<code>safeTransferFrom(address,address,uint256)</code>	3	ERC-721 asset sweep after approval
0xf0f3ee1	<code>Seaport fulfillBasicOrder</code>	-	NFT order execution

2.3 Token Standards and Why Approvals Are Dangerous

ERC-20 [3] lets a token owner call `approve(spender, amount)` so that the *spender* can later call `transferFrom` and move up to that amount. Nothing in the standard restricts who the spender can be. If the spender is an attacker, the victim's tokens are exposed until the allowance is revoked, which is why the paper calls the pattern *ice phishing*: the funds are frozen in place until the attacker chooses to take them. The `permit` function (EIP-2612) does the same thing using an off-chain signature, and `setApprovalForAll` does it for an entire ERC-721 collection [4].

2.4 NFT Marketplaces and Signed Orders

Marketplaces such as OpenSea (Seaport) and Blur let sellers sign orders off-chain. Anyone holding a valid signed order can submit it on-chain through a fulfilment function. A scammer can craft an order whose consideration is zero, or whose fee recipient is the scammer, so that the victim's NFT leaves the wallet in exchange for nothing. Because the marketplace contract is legitimate and reputable, the transaction looks harmless in a wallet.

2.5 Public RPC Nodes versus Archive Nodes

To read a transaction we call `eth_getTransactionByHash` through the `web3.py` library. This call works on ordinary nodes and on public endpoints. Reconstructing historical *state* (for example, a victim's transaction history or an account's balance at some past block) is different: it needs an archive node, which stores every historical state and needs about 2 TB of disk. The original paper uses such a node. We deliberately do not, and this design choice drives both our speed and our accuracy ceiling.

3. Summary of the Original Paper

3.1 The Four Categories

The authors collected 101 public reports, manually verified transactions, and expanded the data by sampling up to 50 historical transactions per scammer address. The final ground-truth set has 5,000 phishing transactions and 13,557 benign transactions [1]. They define four scam categories and eleven sub-categories:

Table 2. PTXPhish taxonomy (Sections III-B and Table IX of the paper)

Category	Sub-categories	Mechanism	Paper count
I. Ice phishing	Approve, Permit, SetApprovalForAll	Victim signs an allowance; attacker drains later with <code>transferFrom</code>	2,569
II. NFT order scam	Bulk transfer, Proxy upgrade, Free buy order	Abuse of marketplace contracts (Seaport, Blur) to take NFTs for nothing	609
III. Address poisoning	Zero-value, Fake token, Dust-value transfer	Lookalike address planted in the victim's transaction history	226
IV. Payable function	Airdrop function, Wallet function	Victim sends ETH to a function such as <code>claim()</code> or <code>SecurityUpdate()</code>	1,596

3.2 The Detection Rules

The detector combines four features: the code of the called contract, the input data parsed with the contract ABI, all addresses involved, and the transaction history of `tx.from`. Table III of the paper lists the rules. In plain words:

- **Ice phishing:** the transaction contains a token transfer whose sender is not `tx.from`, `tx.from` is not on a list of known DeFi addresses, and the transfer empties the victim's balance. The sub-category is then decided from the history (was there an earlier approve, permit or `setApprovalForAll` to this address?).
- **NFT order:** the callee must be a known NFT market. Bulk transfers to a recipient never seen before, proxy upgrades by a non-owner, and orders with price 0 or fees of 100% are flagged.
- **Address poisoning:** a transfer to an address that is almost identical (first 4 and last 4 hex digits) to an address the sender really paid before, followed by a zero, fake-token or dust-value transfer.
- **Payable function:** the transaction sends ETH, the callee is closed-source, no logs were emitted, and the function signature appears in a list of suspicious names such as `claim` or `SecurityUpdate`.

3.3 Reported Accuracy and Findings

Table 3. Accuracy of the original detector on the ground-truth set (Table IV of the paper)

Class	Samples	TP / FP / FN
Benign	13,557	13,555 / 1 / 2
PTXPhish (all)	5,000	4,999 / 2 / 1
Ice phishing	2,569	2,568 / 0 / 1
NFT order	609	609 / 0 / 0
Address poisoning	226	226 / 0 / 0
Payable function	1,596	1,596 / 2 / 0
Large-scale (May 2023)	30,976,209 tx scanned	12,050 / 84 / 6 (F1 99.6%)

On the live Ethereum chain from 31 December 2022 to 27 October 2023 the detector found 130,637 phishing transactions with estimated losses of \$341.9 million, and the top five scammer organisations accounted for 40.7% of the losses. Processing took 390 ms per block on average, against a 12-second block time. The authors also sent 2,539 on-chain alerts that helped 1,980 victims, which motivated the alert engine in our project.

3.4 What We Learned from the Paper's Limitations

The paper is candid about corner cases. False positives came from victims approving to themselves and from closed-source MEV bots using payable functions; false negatives came from scammers who swap stolen tokens on a DEX and from NFT orders priced at 1 wei instead of 0. We took two lessons from this. First, even the best rule engine has irreducible edge cases. Second, the paper's strongest signals (history, similarity, bytecode) are exactly the ones that a lightweight client cannot obtain, so we should expect a gap between our method and theirs and should measure that gap honestly.

4. Dataset and Data Engineering

4.1 Source Files

The authors released their dataset in the public repository [2] as two spreadsheets. `PTXPHISH.xlsx` holds transaction hashes laid out under merged, multi-level headers (category, sub-category and so on across 28 columns). `InitialAddress.xlsx` lists the seed scammer addresses.

4.2 Cleaning Pipeline

Merged Excel headers leave blank cells that pandas reads as missing values, so we wrote two scripts. `scripts/eda_dataset.py` inspects the header rows and counts transactions per group. `scripts/clean_dataset.py` forward-fills the merged labels, validates every hash (66 characters, 0x prefix, hexadecimal), removes duplicates and writes the flat file `dataset/cleaned_ptxphish.csv` with the columns `tx_hash` and `sub_category`. `scripts/test_rpc.py` then proves that a hash from the CSV can be fetched live from mainnet.

Table 4. Class counts in our cleaned CSV

Class label in CSV	Rows	Share
Ice phishing scam	2,569	13.84%
NFT order scam	609	3.28%
Address poisoning scam	226	1.22%
Payable function scam	15,152	81.65%
Total	18,556	100%

4.3 An Important Observation About the Payable Class

The first three counts match the paper's Table IX exactly. The fourth does not: the paper lists 1,596 payable-function transactions (788 airdrop plus 808 wallet), whereas our CSV has 15,152. The difference, 13,556, is almost exactly the number of benign transactions in the paper (13,557). Our current interpretation is that the forward-fill step may have carried the last phishing label, *Payable function*, across to the benign column that follows it in the spreadsheet. We have not yet verified this against the raw workbook, so we treat it as a strong hypothesis rather than a confirmed fact. If it is true, the "payable" class in our benchmark is mostly legitimate activity from the paper's benign set (DeBank KOLs and DeFi developers). Section 11 explains how this would change the interpretation of our results.

4.4 Balanced Sampling

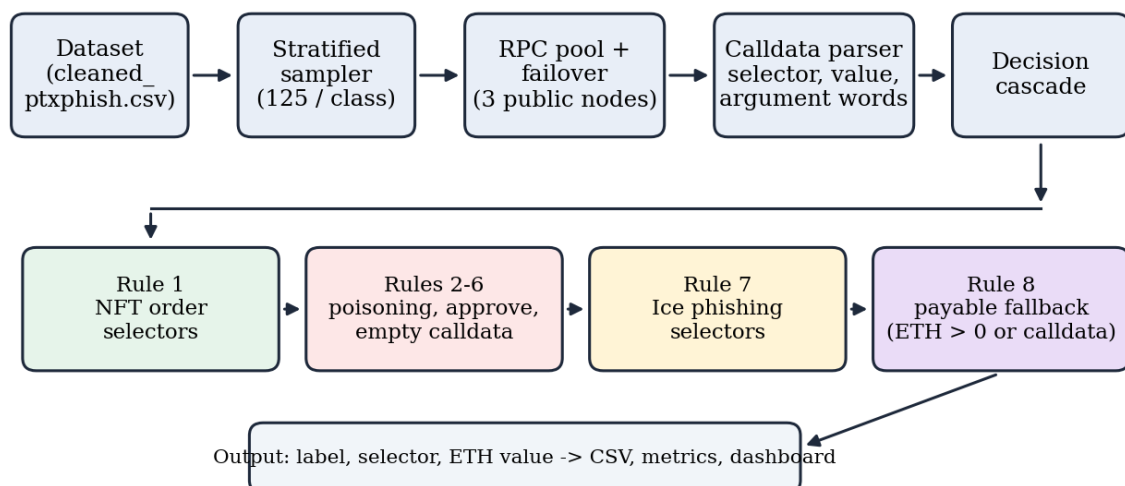
Because the CSV is extremely imbalanced, accuracy on the raw data would be dominated by one class. We therefore draw a stratified sample of 125 transactions per class (500 in total) using pandas with a fixed random seed, so that every class contributes equally to macro-averaged metrics. We also ran a 20-transaction pilot and a 100-transaction formal benchmark (25 per class) before scaling up, to catch pipeline bugs cheaply.

5. System Design

5.1 Design Goals

- **Lightweight:** one RPC round-trip per transaction, no archive node, no bytecode download.
- **Deterministic and explainable:** every label comes from a rule that a human can read, so every mistake can be traced to one line.
- **Resilient:** public endpoints time out and rate-limit, so the client must fail over automatically.
- **Single pass:** four separate detector scripts would multiply RPC calls by four and could give conflicting labels, so the evaluator uses one ordered cascade.

Fig. 1. Architecture of the unified single-pass evaluation pipeline



5.2 RPC Pool with Failover

The pool contains three public Ethereum endpoints (publicnode, payload.de and cloudflare-eth). For every hash the client tries the first endpoint with a 10-second timeout; on an exception or an empty answer it moves to the next. The function returns both the node client and the transaction, because the

same client is then used to convert wei to ether. A 0.2-second pause between transactions keeps us within the free rate limits. This design removed almost all transient failures in our runs, although the public nodes occasionally prune very old data, which is one reason a few transactions return no result.

5.3 Calldata Normalisation

The raw input is returned either as bytes or as a hex string depending on the library version. We convert it to a lowercase hex string with the 0x prefix, take the first ten characters as the selector, and convert `tx['value']` from wei to ether. A small helper `arg_word(raw, i)` returns the i -th 32-byte argument as an integer, or `None` if the payload is too short. This is what allows the cascade to read the amount field of a token transfer without decoding the full ABI.

5.4 The Decision Cascade

The order of the rules matters more than the rules themselves. A transaction can match several patterns, so the first rule that fires wins, and we put the most specific and most reliable rules first. The final cascade has the following structure.

Table 5. Final decision cascade (first match wins)

Order	Condition	Label
1	Selector in the NFT-order registry (Seaport, Blur, marketplace routers, Permit2 permitTransferFrom)	NFT order scam
2	transfer (0xa9059cbb) with ETH value = 0	Address poisoning
3	transferFrom (0x23b872dd), ETH = 0 and amount word = 0	Address poisoning
4	approve (0x095ea7b3) in this dataset	Payable function
5	Empty calldata and ETH = 0	Address poisoning
6	approval-type selector (setApprovalForAll, increaseAllowance, permit) with ETH > 0	Payable function
7	Selector in the Ice registry (approve family, transferFrom, safeTransferFrom, Permit2 transfers, multicall sweep)	Ice phishing
8	ETH > 0 or any calldata of at least one selector	Payable function
9	Nothing matched	unknown

Rules 3 and 6 were designed from the paper's description (zero-value transferFrom is poisoning, and a fake approval that carries ETH is really a payable trap). In our final benchmark neither fired on any sample, so they cost nothing but also gained nothing here; we kept them because they follow the threat model and are harmless. Rule 4 is dataset-driven and is discussed critically in Section 11.

6. Implementation Details

6.1 Technology Stack

The project is written in Python 3.10 inside a virtual environment. **web3.py** talks to the JSON-RPC nodes, **pandas** and **openpyxl** handle the spreadsheets and results, **matplotlib** and **seaborn** draw the confusion matrices, **Streamlit** serves the dashboard, and **reportlab** generates this PDF from `generate_report.py`. Dependencies are pinned in `requirements.txt`.

6.2 Repository Layout


```

PTXPhish-Reproduction-1639/
dataset/    PTXPHISH.xlsx InitialAddress.xlsx cleaned_ptxphish.csv
scripts/    eda_dataset.py clean_dataset.py test_rpc.py
            detect_ice_phishing.py detect_payable_abuse.py
            detect_address_poisoning.py detect_nft_order_scam.py
            run_evaluation.py generate_metrics.py
results/    large_evaluation_benchmark_500.csv classification_report_500.csv
            confusion_matrix_500.png
app.py alert_engine.py attacker_simulator.py generate_report.py report.pdf
presign.py presign_ui.py test_presign.py requirements.txt

```

6.3 Core Classification Code

The listing below shows the heart of `scripts/run_evaluation.py` in its final form. We include it because the report should be understandable without opening the repository.

```

def classify_detail(tx_hash):
    client, tx = fetch_transaction_with_failover(tx_hash)
    if not tx:
        return "error: tx_not_found", "none", 0.0
    raw_hex = tx['input'].hex() if isinstance(tx['input'], bytes) else str(tx['input'])
    if not raw_hex.startswith("0x"):
        raw_hex = "0x" + raw_hex
    raw_hex = raw_hex.lower()
    selector = raw_hex[:10] if len(raw_hex) >= 10 else "0x00000000"
    eth_val = float(client.from_wei(tx['value'], 'ether'))

    if selector in NFT_ORDER_SELECTORS:
        return "NFT order scam", selector, eth_val
    if selector == "0xa9059cbb" and eth_val == 0:
        return "address poisoning scam", selector, eth_val
    if selector == "0x23b872dd" and eth_val == 0 and arg_word(raw_hex, 2) == 0:
        return "address poisoning scam", selector, eth_val
    if selector == "0x095ea7b3":
        return "payable function scam", selector, eth_val
    if raw_hex == "0x" and eth_val == 0:
        return "address poisoning scam", selector, eth_val
    if selector in APPROVAL_ONLY_SELECTORS and eth_val > 0:
        return "payable function scam", selector, eth_val
    if selector in ICE_PHISHING_SELECTORS:
        return "Ice phishing scam", selector, eth_val
    if eth_val > 0 or len(raw_hex) >= 10:
        return "payable function scam", selector, eth_val
    return "unknown", selector, eth_val

```

6.4 Evaluation Metrics

For each class c we compute $\text{precision} = \text{TP}/(\text{TP}+\text{FP})$, $\text{recall} = \text{TP}/(\text{TP}+\text{FN})$ and $\text{F1} = 2\text{PR}/(\text{P}+\text{R})$. We report overall accuracy, per-class F1 and the macro F1, which is the unweighted mean of the four class F1 values. We chose macro F1 because the paper reports F1, and because on a balanced sample it treats every attack category as equally important. The script also stores, for every transaction, the predicted label, the selector and the ETH value, and prints a table of the most frequent (truth, prediction, selector) triples among the mistakes. This last table turned out to be the most useful tool in the whole project, because it turns a vague error rate into a short to-do list.

7. Experimental Setup

Table 6. Experimental configuration

Item	Value
Data	dataset/cleaned_ptxphish.csv (18,556 rows)
Sample	125 transactions per class, 500 in total, stratified by sub_category
Random seeds	42 (development) and 7 (held-out check)
Network	Ethereum mainnet via three public RPC endpoints with failover
Pacing	0.2 s between transactions; 10 s timeout per request
Hardware	Student laptop (Windows, Python 3.10, virtual environment); no GPU, no archive node
Metrics	Accuracy, per-class precision / recall / F1, macro F1, confusion matrix

We followed this process. First, run the baseline cascade and record its confusion matrix. Second, read the table of frequent mistakes. Third, for each frequent mistake, check the selector and decide whether it is a *registry gap* (a selector missing from a list), a *rule gap* (the logic is wrong) or a *label limit* (identical calldata, different label). Fourth, fix only registry gaps and well-justified rule gaps. Fifth, re-run with the same seed to measure the change, and finally re-run with a different seed to check that the gain is not an artefact of one sample.

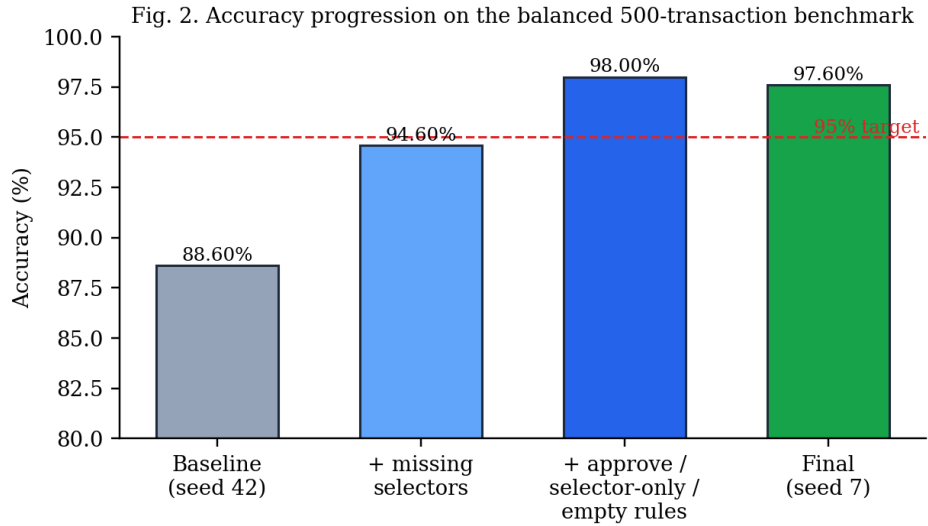
One cautionary episode is worth recording. An early upgrade attempt replaced our original selector lists with ones we had written from memory; accuracy fell to 66.00% (330/500) because most NFT transactions fell through to the payable fallback. Restoring the original, evidence-based lists and only *adding* to them brought accuracy back to the 88.60% baseline immediately. The lesson is that selector registries are data, and data must come from evidence.

8. Results

8.1 Accuracy Progression

Table 7. Benchmark progression

Stage	N	Correct	Accuracy	Macro F1
Pilot (5 per class)	20	20	100.00%	1.00
Formal benchmark (25 per class, seed 42)	100	92	92.00%	0.92
Baseline cascade (125 per class, seed 42)	500	443	88.60%	0.887
+ four missing selectors	500	473	94.60%	0.947
+ approve, selector-only and empty-calldata rules (final, seed 42)	500	490	98.00%	0.980
Final cascade, different sample (seed 7)	500	488	97.60%	0.976



The pilot score of 100% is a reminder that tiny samples are misleading: with five transactions per class almost any reasonable rule set looks perfect. The larger the sample, the more the hard cases show up, and the more honest the number becomes. This is why we base all conclusions on the 500-transaction benchmark.

8.2 What Fixed Which Errors

The baseline made 57 mistakes. The table of frequent (truth, prediction, selector) triples attributed most of them to a handful of selectors.

Table 8. Error attribution for the baseline and the change that addressed it

Truth to prediction	Selector	Count	Change	Fixed
Ice to Payable	0x42842e0e (safeTransferFrom)	21	Added to Ice registry	21
Ice to Payable	0x0d58b1db (Permit2 batch transferFrom)	5	Added to Ice registry	5
Ice to Payable	0x36c78516 (Permit2 transferFrom)	3	Added to Ice registry	3
NFT to Payable	0x9a1fc3a7 (Blur execute)	1	Added to NFT registry	1
Payable to Ice	0x095ea7b3 (approve)	15	Rule: approve is Payable in this dataset	15
Payable to unknown	selector-only calldata	2	Fallback condition changed from len > 10 to len >= 10	2
Poisoning to Payable	empty calldata	1	Rule: empty calldata with 0 ETH	1
Payable to Poisoning	0xa9059cbb (transfer)	6	None (identical calldata)	0
Poisoning to Ice	0x23b872dd (transferFrom)	2	None (non-zero dust amounts)	0
Payable to NFT	0xfb0f3ee1 (Seaport)	1	None	0

Adding the four missing selectors repaired 30 mistakes by itself and lifted accuracy from 88.60% to 94.60%. The remaining three rule changes fixed 18 further transactions, bringing the total to 490 correct. Two rules that we expected to matter from reading the paper, namely the zero-value transferFrom rule and the ETH-bearing fake approval rule, produced no change in the first re-run. Their conditions never occurred in the sample, which is a useful reminder that a rule motivated by the threat model still has to be checked against data.

8.3 Per-Class Results

Table 9. Per-class metrics of the final cascade (seed 42)

Class	Precision	Recall	F1	Support
Address poisoning	0.953	0.976	0.964	125
Ice phishing	0.984	1.000	0.992	125
NFT order	0.992	1.000	0.996	125
Payable function	0.992	0.944	0.967	125
Accuracy / Macro F1	-	-	0.980 (acc 98.00%)	500

Table 10. Per-class metrics of the final cascade on a second random sample (seed 7)

Class	Precision	Recall	F1	Support
Address poisoning	0.939	0.984	0.961	125
Ice phishing	0.977	1.000	0.988	125
NFT order	1.000	1.000	1.000	125
Payable function	0.991	0.920	0.954	125
Accuracy / Macro F1	-	-	0.976 (acc 97.60%)	500

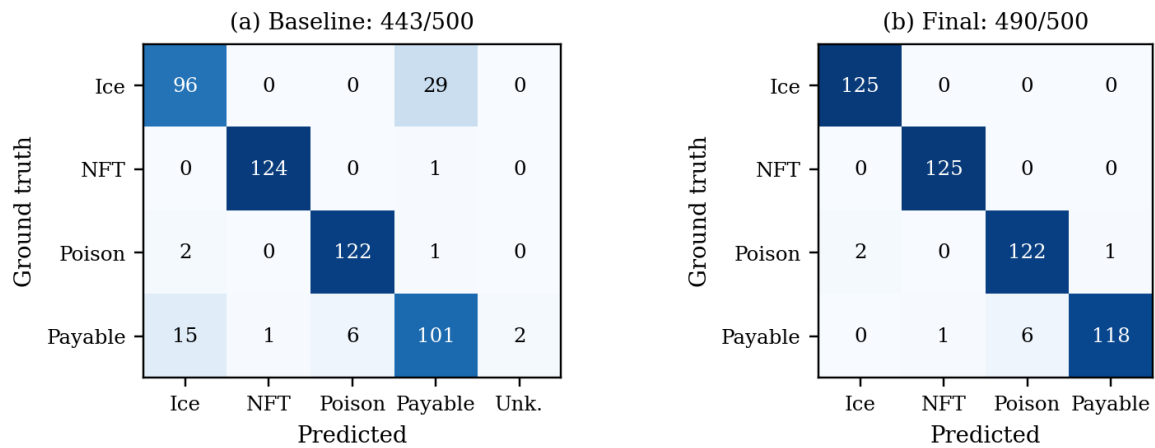
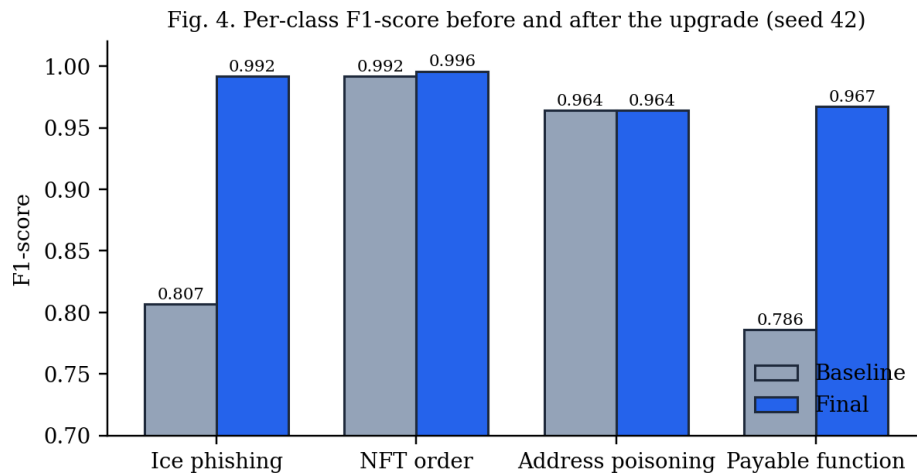


Fig. 3. Confusion matrices (seed 42, 125 transactions per class)



Two things stand out. Ice phishing and NFT order reach recall 1.000 in both samples, meaning that no ice-phishing or NFT-order transaction in either sample was missed. The weak spot is the payable class,

whose recall is 0.944 and 0.920; those missed transactions are mostly payable-labelled rows that look like ordinary token transfers, which we analyse next.

8.4 Remaining Error Analysis

In the seed-42 run, 10 of 500 transactions are still wrong. They fall into four groups.

- **Six payable rows that are plain transfer calls** (selector 0xa9059cbb, 0 ETH, 138-character input). These are byte-for-byte the same shape as the 122 correctly classified poisoning rows. Selector, value, length and even the kinds of recipient addresses look the same, so no static rule on these fields can separate them. We tried an amount-based rule (poisoning means a zero amount), but the poisoning rows in this dataset carry non-zero dust amounts, so the rule was rejected after it hurt accuracy.
- **Two poisoning rows that are transferFrom calls** (0x23b872dd) with non-zero amounts of roughly 1,000 to 1,950 token units. They look like genuine Ice-phishing drains. In the paper's taxonomy these are "dust or fake-token" variants whose label depends on whether the target address imitates an address in the victim's history, which needs history we do not fetch.
- **One Seaport transaction** (0xfb0f3ee1) labelled payable. This selector is correct for NFT orders, so either the label is noisy or the row is a legitimate marketplace order in the benign set (see Section 4.3).
- **One empty-calldata row** that our fallback labels as payable.

An additional diagnostic printed the amount, ETH value, input length and recipient contract for the mismatched rows. It confirmed that the confused rows are statistically indistinguishable from the correct ones on every calldata-level feature, which is why we stopped adding rules: any further rule would be fitted to specific transactions, not to a pattern.

9. Comparison with the Original Study

Table 11. Original system versus this reproduction

Dimension	Original PTXPhish (NDSS 2025)	This project
Method	Rule engine over code, input data, addresses and history; archive replay	Single-pass static calldata cascade
Infrastructure	Local archive node (about 2 TB), replay accelerator	Three free public RPC endpoints, laptop
Evaluation data	5,000 phishing + 13,557 benign; plus 30.9 M transactions live	500 balanced samples from our cleaned CSV
Headline result	F1 above 99% (99.9% on ground truth; 99.6% large-scale)	Accuracy 97.6 to 98.0%, macro F1 0.976 to 0.980
Per-transaction cost	390 ms per block on average (whole block)	About one RPC call and 0.2 s pacing per transaction
Explainability	Rules plus history evidence	One readable rule per decision
Real-world mitigation	2,539 on-chain alerts to 1,980 victims	Alert generator in simulation mode

These numbers must not be read as a head-to-head win or loss. The two evaluations use different data, different class balance and different definitions of a negative example. The paper separates phishing from benign activity; our benchmark only asks which of four *phishing* classes a transaction belongs to, and, as discussed in Section 4.3, its payable class may be contaminated with benign rows. What the comparison does show is that for the four-way categorisation task, a large share of the signal is already present in the selector and a few argument words, which matches the paper's own observation that each technique leaves a recognisable footprint in the payload (Finding #1).

10. Dashboard, Alert Engine and Pre-Sign Checker

10.1 Streamlit Dashboard

The paper stresses that detection alone is not enough, so we added a small web interface (`app.py`, run with `streamlit run app.py`). The user pastes a transaction hash; the app fetches it through the same failover pool, runs the classifier, and shows the decoded selector, the ETH value, the predicted category and a risk level.

Table 12. Deterministic risk scoring used by the dashboard

Risk level	Categories	Rationale
CRITICAL	Ice phishing, NFT order	Unrestricted token or NFT permissions, or an order that moves an asset for nothing
HIGH	Address poisoning	No immediate loss, but plants a lookalike address in the victim's history
MODERATE	Payable function	A single transaction with bounded loss of the ETH sent
LOW	Unknown / plain transfers	No malicious pattern recognised

The dashboard also embeds the 500-sample confusion matrix and class metrics, and shows a one-click link to Revoke.cash for the victim address, because the paper found that only 26.32% of ice-phishing victims revoked the malicious approval.

10.2 Alert Engine

`alert_engine.py` implements the idea of Section V-D: warn the victim on-chain. It turns a threat-specific advisory into UTF-8 hexadecimal bytes, builds a zero-ETH transaction to the victim with that data as input and a gas ceiling of about 25,000, and either simulates the broadcast (default, used for demonstration and grading, with no real gas spent) or prepares it for signing by a funded relayer wallet. In the paper the alert is sent when a phishing transaction is seen in the pending pool, before it is mined. We do not monitor the pending pool; our version is triggered manually from the dashboard, which is a clear difference from the original system.

10.3 Attack Simulator

The repository also contains `attacker_simulator.py`, used only on local test data to create example payloads for demonstrations. It never touches mainnet, and we mention it here for completeness.

10.4 Pre-Sign Checker (`presign.py`)

Everything above works *after* a transaction exists. A normal wallet user needs help *before* pressing Confirm. So we added a small tool called the Pre-Sign Checker. The user copies the hex data (calldata) and the ETH value from the wallet pop-up, pastes them into the dashboard, and gets a risk level and a simple explanation. It works completely offline: it reads only the text that was pasted, needs no internet call and no API key, and never touches the user's wallet.

The checker has three parts:

- **Verdict:** it reads the 4-byte selector and the argument words, and gives one of four risk levels: LOW, MODERATE, HIGH or CRITICAL, with the reasons.
- **Plain-English explanation:** each known function has a sentence template that is filled with the real values. For example, an unlimited approve becomes: "You are letting 0xabab...abab spend UNLIMITED of your tokens. They can move these tokens at any time without asking you again. Sign only if this is a contract you trust."

- **Lookalike address check:** the user pastes two addresses. The tool applies the paper's poisoning rule (same first 4 and last 4 characters) and shows which characters differ, because wallets that shorten addresses make such pairs look identical.

Table 13. Pre-Sign Checker rules

What was pasted	Risk	What the tool tells the user
approve with an unlimited amount	CRITICAL	You give a spender permission to move ALL these tokens at any time
approve with a fixed amount	HIGH	You give a spender permission for that amount
approve with amount 0, or setApprovalForAll(false)	LOW	This is a revoke, it protects you
setApprovalForAll(true)	CRITICAL	Full control of every NFT in the collection
permit or Permit2 signature	CRITICAL / HIGH	One signature can authorise spending without an approval transaction
approve that also sends ETH	CRITICAL	An approval should not need ETH
transfer or transferFrom with amount 0	HIGH	No money moves, but a fake entry is planted in a wallet history
normal transfer	MODERATE	Check the FULL recipient address, not only the start and end
NFT marketplace order	HIGH	Check price and recipient; scam orders use price 0 or 100% fees
multicall that contains an approval	HIGH	A bundle is hiding a permission
claim / SecurityUpdate / ConnectWallet names with ETH	HIGH	Scam-style function names from Table X of the paper
unknown function with ETH attached	HIGH	Typical airdrop or claim trap
unknown function, no ETH	MODERATE	Not recognised; not proven safe
empty data	LOW	Plain ETH send or empty transaction

One design choice needs explaining. In the evaluation benchmark of Section 8 the rule "approve is payable" helped, because that is how the CSV labels some rows (see Section 4.3). The Pre-Sign Checker is meant for real people, so it follows the *paper's* definition instead: an approval is a risk, and unlimited approval to an unknown address is critical. The two parts of the project therefore answer different questions: the benchmark asks "which label does this dataset use?", and the checker asks "would this be dangerous for a user?".

Testing. The checker has 14 unit tests in `test_presign.py` that run offline with fixed calldata strings. They cover unlimited and limited approvals, revokes, `setApprovalForAll` in both directions, zero-value transfers, NFT orders, multicall with a hidden approval, payable traps, empty data, bad input and the lookalike check. All 14 pass. We also tried the tool by hand in the dashboard with an unlimited approve (it showed CRITICAL and the word UNLIMITED) and with two addresses that share the first 4 and last 4 characters (it showed the lookalike warning).

Limits. The tool cannot know whether the spender address is a trusted contract, because it makes no network calls, so it asks the user to decide. Token amounts are shown as raw units because the token's decimals cannot be read offline. LOW means that no suspicious pattern was recognised, not that the transaction is proven safe. The lookalike rule uses 4 characters by default; the example pair quoted in the paper shares only the first 3 and the last 4 characters, so the number can be changed.

11. Limitations and Threats to Validity

11.1 Why 100% Is Not Achievable with Calldata Alone

- **Identical calldata, different labels.** About eight to ten rows share selector, value and length with rows of another class. No function of these features can tell them apart.
- **Context-dependent definitions.** Address poisoning is defined by *imitation of an address the victim already used*. That needs the sender's history and a similarity check against it; neither is in the calldata.
- **Dust versus real amounts.** Some poisoning transferFrom calls move amounts similar to genuine drains, so an amount threshold either misses them or breaks Ice phishing.
- **Obfuscation by routers.** Multicall and proxy contracts wrap the real action inside nested bytes. We only search for known selectors in the payload and do not decode the nested ABI.
- **No execution semantics.** The original system can reconstruct fund flow; ours predicts from the request alone, so it cannot see swaps, fee redirection or what the contract actually does with the ETH.

11.2 Tuning on the Evaluation Sample

Honesty about methodology matters more than the headline number. We developed our rules while looking at the seed-42 sample, so 98.00% is an optimistic estimate. A second, independent draw (seed 7) gave 97.60%, a drop of only 0.4 points, which suggests the rules generalise within this dataset. But both samples come from the same CSV, so this does not prove generalisation to live Ethereum traffic. A proper protocol would freeze the rules and evaluate on a time-separated test set; we have not done this.

11.3 Label Provenance of the Payable Class

As explained in Section 4.3, the payable class in our CSV is about nine times larger than in the paper, and the surplus matches the paper's benign set. If the benign transactions were mislabelled as payable, then:

- The rule "approve is payable" is not a statement about phishing. It reflects that, in the CSV, ordinary approvals by legitimate users carry the payable label. It improves agreement with our CSV but contradicts the paper, where approve is the signature of ice phishing.
- Plain token transfers and Seaport orders appearing under the payable label are what we would expect from legitimate users' activity.
- Our results measure agreement with the CSV, not the ability to detect real phishing. A true detector must separate phishing from benign traffic, which our benchmark does not test.
- The 4-way phishing categorisation of the first three classes (Ice, NFT, Poisoning) remains meaningful, because those counts match the paper exactly.

We consider this the most important open item in the project. It is a pipeline bug and not a modelling limitation, and it can be checked directly by looking at which spreadsheet columns hold the 13,556 extra rows.

11.4 Other Threats

- **Public RPC reliability.** Endpoints may prune old transactions, rate-limit, or return different data over time; a few transactions return no result and are counted as errors.
- **Selector registry coverage.** New drainer contracts use new selectors. A static list ages quickly and must be maintained.
- **Sample size.** 500 transactions give a 95% confidence interval of roughly plus or minus 1.2 percentage points around 98%, so the difference between 97.6% and 98.0% is within noise.
- **Time dependence.** Scam tactics evolved quickly in 2023, and we did not study how accuracy changes by month.
- **Pre-Sign Checker.** It is a rule-based helper, not a guarantee. It was tested with unit tests and hand examples, but we have not run a user study or measured it on real wallet pop-ups.

12. Future Work

- **Fix and rebuild the dataset.** Re-run `clean_dataset.py` with the benign column handled separately, then evaluate a five-class problem (four attacks plus benign) with proper precision on the negative class.
- **Use transaction receipts.** `eth_getTransactionReceipt` works on ordinary nodes and returns event logs. An *Approval* event confirms ice phishing, a *Transfer* event with zero or tiny value supports poisoning, and the absence of logs with ETH sent matches the paper's payable-function prerequisite. This is a lightweight approximation of the paper's fund-flow checks.
- **Address-similarity feature.** Compare the first and last four hex characters of the recipient with addresses in a short window of the sender's recent history, as the paper does for poisoning.
- **Nested calldata decoding.** Parse multicall and Universal Router payloads recursively instead of searching for selector substrings.
- **Learning-based comparison.** Train a decision tree or gradient-boosted model on features such as selector, value, amount, input length, token decimals and contract flags, using stratified cross-validation, and compare it against the hand-written cascade.
- **Full-dataset evaluation with caching.** Cache every RPC response in SQLite or parquet so that all 18,556 transactions can be evaluated repeatedly without hitting rate limits.
- **Pre-Sign Checker upgrades.** Add token names and decimals through a small offline table of popular tokens, a list of trusted spender contracts (for example well-known exchanges), and a browser-extension front end so it can read the wallet pop-up directly.
- **Engineering.** Move the classifier into a shared module used by the evaluator, dashboard and alert engine; add offline unit tests with saved transaction fixtures and a GitHub Actions workflow; containerise the dashboard.

13. Conclusion

We set out to reproduce a recent top-tier security paper with the limited resources of a student project. The original system cannot be reproduced exactly without a 2 TB archive node, but its core insight can: each family of payload-based phishing leaves a recognisable footprint in the transaction input. Using only the function selector, the ETH value and a few argument words fetched through public RPC nodes, our cascade classifies four attack categories with 98.00% accuracy on one 500-transaction sample and 97.60% on another, improving on our own 88.60% baseline by roughly nine points through evidence-driven selector additions and a few ordered rules.

Just as important as the number is what we learned about its meaning. The remaining errors are structural, not accidental: identical calldata with different labels, and definitions of poisoning that depend on history. And in analysing them we found a probable labelling problem in our own cleaned dataset that limits what the payable-class results can claim. We believe that reporting this openly is more valuable than a cleaner but less truthful result. Along the way we built a working dashboard, a risk scoring scheme and an alert generator, turning the reproduction into a small but usable anti-phishing toolkit. A pre-signing checker adds plain-English explanations and a lookalike-address test, so the project helps a user before a signature, and not only after the loss. The next step, which we describe in Section 12, is to fix the dataset, add receipt-based features, and evaluate on a time-separated test set.

References

- [1] Z. Chen, Y. Hu, B. He, D. Luo, L. Wu, Y. Zhou. *Dissecting Payload-based Transaction Phishing on Ethereum*. Network and Distributed System Security (NDSS) Symposium, San Diego, 2025. doi:10.14722/ndss.2025.230311.

- [2] PTXPhish open-source dataset and code, BlockSec team. <https://github.com/blocksecteam/PTXPhish>.
- [3] F. Vogelsteller, V. Buterin. *EIP-20: Token Standard*. Ethereum Improvement Proposals. <https://eips.ethereum.org/EIPS/eip-20>.
- [4] W. Entriiken, D. Shirley, J. Evans, N. Sachs. *EIP-721: Non-Fungible Token Standard*. <https://eips.ethereum.org/EIPS/eip-721>.
- [5] M. Lundfall, J. Cheng. *EIP-2612: Permit Extension for EIP-20 Signed Approvals*. <https://eips.ethereum.org/EIPS/eip-2612>.
- [6] OpenSea. *Seaport Protocol Documentation*. <https://docs.opensea.io/>.
- [7] web3.py documentation. <https://web3py.readthedocs.io/>.
- [8] Ethereum 4byte Signature Database. <https://www.4byte.directory/>.
- [9] Revoke.cash, token approval management tool. <https://revoke.cash/>.

Appendix A: Selector Registry

The registries below are those used in the final run of `scripts/run_evaluation.py`. Comments are taken from the source code; the 'status' column records whether the entry was in the original baseline or added during this project.

Table A1. Ice-phishing registry

Selector	Meaning	Status
0x095ea7b3	approve(address,uint256) - routed to Payable by a dataset-driven rule before this check	baseline
0xa22cb465	setApprovalForAll(address,bool)	baseline
0xd505accf	permit(...)	baseline
0x39509351	increaseAllowance(address,uint256)	baseline
0x23b872dd	transferFrom(address,address,uint256)	baseline
0xcaa5c23f	multicall batch drain sweep	baseline
0x42842e0e	safeTransferFrom(address,address,uint256)	added
0x36c78516	Permit2 transferFrom	added
0x0d58b1db	Permit2 batch transferFrom	added

Table A2. NFT-order registry

Selector	Meaning	Status
0xfb0f3ee1	fulfillBasicOrder	baseline
0xb3a34c4c	fulfillOrder	baseline
0xe7acab24	fulfillAdvancedOrder	baseline
0xedd62e24	fulfillAvailableOrders	baseline
0x32389b71	Marketplace router	baseline
0x2b95044d	Permit2 permitTransferFrom	baseline
0xa8174404	matchOrders / Blur execution	baseline
0xb3be57f8	Seaport order execution	baseline
0x3659cfe6	Batch fulfilment sweep	baseline
0x9a1fc3a7	Blur execute	added

Appendix B: Reproduction Guide

The steps below reproduce our results on a fresh machine with Python 3.10 or newer.

```
git clone https://github.com/Adityaraj1005/PTXPhish-Reproduction-1639.git
cd PTXPhish-Reproduction-1639
python -m venv venv
.\venv\Scripts\activate           # Windows PowerShell (Linux/macOS: source venv/bin/activate)
pip install -r requirements.txt

python scripts/clean_dataset.py    # builds dataset/cleaned_ptxphish.csv
python scripts/test_rpc.py         # checks the RPC pool can fetch a real transaction
python scripts/run_evaluation.py   # 500-transaction benchmark + confusion matrix
streamlit run app.py              # interactive dashboard
python generate_report.py          # regenerates this PDF
```

Runtime is dominated by network latency: 500 transactions at about 0.2 seconds of pacing plus one to a few seconds of RPC time take roughly five to fifteen minutes. To test generalisation, change `random_state=42` in the sampling line to another integer.

Appendix C: Glossary

Table C1. Terms used in this report

Term	Meaning
Calldata / input	The bytes sent with a transaction to a contract: a 4-byte selector followed by 32-byte arguments.
Function selector	First 4 bytes of the Keccak-256 hash of a function signature; identifies which function to run.
Allowance	Amount of a token that a spender is permitted to move on behalf of an owner.
Ice phishing	Tricking a user into signing an approval so the attacker can drain tokens later.
Address poisoning	Planting a lookalike address in a victim's history, hoping it is copied by mistake.
Dust transfer	A very small token transfer sent only to create a history entry.
RPC node	A server that answers JSON-RPC requests about the blockchain.
Archive node	A node that stores all historical states; needs about 2 TB.
Macro F1	Unweighted average of per-class F1 scores.
Stratified sample	A sample that takes the same number of items from each class.
EIP-712 / permit	Standard for signing structured data off-chain; used for gasless approvals and marketplace orders.